

Chatbot (Cohere) — Documentation

Ce document décrit en détail le chatbot intégré à l'application Qt (dialog `ChatbotDialog`). Il explique l'architecture, les fichiers sources, les dépendances, les variables d'environnement requises, le format des requêtes/réponses, la gestion d'erreurs et des conseils pour le débogage.

Résumé

- **But:** Fournir une interface de chat simple utilisant l'API Cohere pour générer des réponses textuelles.
- **Emplacement du code:** les fichiers principaux sont `chatbot.h` et `chatbot.cpp` (nouveau dialog), avec des stubs allégés dans `openai_chatbot.h` / `openai_chatbot.cpp` pour compatibilité.
- **Endpoint utilisé:** <https://api.cohere.ai/v1/chat>.
- **Sécurité:** la clé API n'est pas en dur dans le code. Utiliser la variable d'environnement `COHERE_API_KEY`.

Fichiers clés

- `chatbot.h` : déclaration du `ChatbotDialog` (QDialog). UI minimale (zone d'affichage, champ d'entrée, bouton envoyer, bouton effacer).
- `chatbot.cpp` : implémentation de la UI, envoi de requêtes via `QNetworkAccessManager`, parsing des réponses, et gestion d'erreurs.
- `fournisseur.cpp` : point d'intégration du bouton qui ouvre le `ChatbotDialog` (par ex. `connect(ui->chabot, &QPushButton::clicked, ...)`).
- `openai_chatbot.h`, `openai_chatbot.cpp` : versions stubs (remplacées pour enlever le heavy system prompt et le CRUD complexe).
- `interface.pro` : ajoute `network` et `printsupport` ; contient `SOURCES/HEADERS` nécessaires.

Comment ça marche (flux)

1. L'utilisateur ouvre le dialogue de chat (bouton dans l'UI principale).
2. `ChatbotDialog` construit le message de l'utilisateur et appelle `onSendClicked()`.
3. `onSendClicked()` lit la clé via `QProcessEnvironment::systemEnvironment()` (`COHERE_API_KEY`) et le modèle (optionnel via `COHERE_MODEL`, sinon valeur par défaut).
4. Le dialog envoie une requête POST JSON à <https://api.cohere.ai/v1/chat> avec les en-têtes:
 - `Authorization: Bearer <COHERE_API_KEY>`
 - `Content-Type: application/json`
 - `Cohere-Version: 2022-12-06` (ajouté par le client)
5. Le serveur Cohere répond. Le code gère plusieurs formes de réponse (voir section suivante).
6. La réponse est affichée dans la zone de chat; en cas d'erreur le dialog affiche un message explicite (statut HTTP, message serveur, conseils).

Variables d'environnement

- `COHERE_API_KEY` : **obligatoire**. Clé API Cohere (ne jamais la committer).

- `COHERE_MODEL` : optionnel. Nom du modèle Cohere à utiliser. Si absent, le code utilise `command-a-03-2025` par défaut.

Exemples (PowerShell temporaire pour lancer l'application) :

```
$Env:COHERE_API_KEY = "<VOTRE_CLE_COHERE>"
$Env:COHERE_MODEL = "command-a-03-2025" # optionnel
qmake
mingw32-make
```

Pour définir la variable de façon persistante sous Windows (PowerShell) :

```
setx COHERE_API_KEY "<VOTRE_CLE_COHERE>"
setx COHERE_MODEL "command-a-03-2025"
# Fermer/re-ouvrir la session pour que setx prenne effet.
```

Format de requête envoyé (JSON)

Exemple de corps JSON envoyé au endpoint `/v1/chat` :

```
{
  "model": "command-a-03-2025",
  "message": "Bonjour, pouvez-vous me donner un résumé ?",
  "max_tokens": 150,
  "temperature": 0.6
}
```

(Remarques : le nom exact du champ et du format dépend de l'implémentation dans `chatbot.cpp`.
L'implémentation actuelle envoie `model` et `message` avec quelques options.)

Formes de réponse et parsing

Cohere peut renvoyer différentes formes JSON selon l'endpoint/version :

- Ancienne forme `generations` : tableau de générations contenant le texte.
- Nouvelle forme `message` : objet contenant `content` (ou `message.content` selon version).
- Parfois `text` ou `outputs` peuvent apparaître.

Le code de `chatbot.cpp` tente d'extraire la réponse en essayant dans l'ordre commun :

1. `generations` → extraire le premier `text`.
2. `message.content` → agréger contenu (si présent).
3. `text` ou `outputs` comme fallback.
4. Sinon, afficher le corps brut renvoyé (pour faciliter le debug).

Gestion d'erreurs

- Si la réponse HTTP a un code ≥ 400 , le dialog affiche le code et tente d'extraire un champ `message` dans le JSON retourné par le serveur.
- Cas fréquent : `404 model not found` → message serveur indiquant que le modèle demandé n'est pas accessible pour le compte. Le dialog propose alors :
 - vérifier `COHERE_MODEL`,
 - utiliser le modèle par défaut (`command-a-03-2025`),
 - ou vérifier les permissions/accès au modèle dans le dashboard Cohere.
- Si `COHERE_API_KEY` est absent, une boîte de dialogue prévient l'utilisateur et l'envoi est annulé.

Débogage et inspection

- Pour voir la requête/réponse exacte, vous pouvez ajouter des logs temporaires dans `chatbot.cpp` :
 - `QDebug()` du JSON envoyé et du texte reçu (après `reply->readAll()`).
- Vérifier le code HTTP retourné (`reply->attribute(QNetworkRequest::HttpStatusCodeAttribute)`).
- Si vous obtenez des erreurs réseau répétées, vérifier :
 - variables d'environnement
 - accès Internet depuis l'application (firewall)
 - si le endpoint attendu `https://api.cohere.ai/v1/chat` n'a pas changé

Construction / Exécution (rappel)

- Re-générer le Makefile et builder :

```
qmake
mingw32-make
```

- Lancer l'exécutable généré depuis l'IDE ou l'explorateur.

Conseils d'amélioration

- Ajouter une sélection de modèle en UI (`QComboBox`) qui écrit `COHERE_MODEL` dans un fichier de config ou `QSettings`.
- Remplacer l'ajout de la ligne « Réponse en cours... » par un indicateur d'état unique (barre de progression ou label) pour éviter d'encombrer la zone de chat.
- Supporter la pagination/scroll asynchrone si les réponses sont longues.
- Ajouter une option pour sauvegarder l'historique de chat localement.

Foire aux questions (FAQ rapide)

- Q : J'ai `model not found` — que faire ?
 - A : Assurez-vous que `COHERE_MODEL` est défini sur un modèle auquel votre compte a accès (par ex. `command-a-03-2025` ou `command-r7b-12-2024`). Sinon, retirez la variable pour

utiliser la valeur par défaut.

- Q : Comment éviter d'exposer la clé ?
 - A : Ne commitez jamais `COHERE_API_KEY`. Utilisez les variables d'environnement ou un store de secrets. En production, utilisez un backend pour proxyfier les requêtes et masquer la clé côté serveur.

Questions / prochaines étapes

- Voulez-vous que j'ajoute une UI pour sélectionner le modèle et sauvegarder le choix ?
- Voulez-vous que j'ajoute du logging (fichier) pour les requêtes/réponses lors du debug ?

Fichier créé : `CHATBOT.md` (à la racine du repo). Si vous souhaitez que je mette ce fichier dans un dossier `docs/`, dites-le et je le déplacerai.